



INF8770 : Technologies Multimédias

TP1 : Compression sans perte de textes

Automne 2026

Équipe :

Benjamin Dupuis (Matricule : 2557585)

Gaël Lienard (Matricule : 2489319)

Chargé de laboratoire :

À compléter

Département de génie informatique et génie logiciel
Polytechnique Montréal

Date de remise : 28 septembre 2026

Table des matières

1	Introduction	2
2	Analyse exploratoire	2
2.1	Question 1 : structure et défis de compression	2
2.2	Question 2 : pistes et choix de méthode	3
3	Implémentation et évaluation	3
3.1	Question 3 : démarche et outils	3
3.2	Question 4 : résultats et discussion	4
4	Conclusion	5

1 Introduction

La compression sans perte doit réduire la représentation d'un fichier tout en permettant de restituer exactement chacun de ses octets. Ce TP compare une référence de Huffman à une méthode prédictive suivie du codage de Golomb, sur trois textes de nature différente : un programme Python, un extrait littéraire français et un tableau météorologique CSV. Les octets sont lus en mode binaire afin que les accents, fins de ligne et autres détails de représentation soient préservés.

L'hypothèse explorée est qu'une prédiction différentielle peut transformer les répétitions locales en petits résidus, ensuite bien adaptés à Golomb. Les résultats disponibles ne confirment toutefois pas l'objectif imposé par l'énoncé : avec les estimations calculées dans le notebook, cette chaîne ne dépasse pas Huffman sur les trois fichiers. Cette limite est analysée explicitement plutôt que masquée.

2 Analyse exploratoire

2.1 Question 1 : structure et défis de compression

Les trois fichiers ont une distribution de symboles non uniforme : leur entropie d'ordre zéro est comprise entre 4,368 et 4,969 bits par octet, au-dessous des 8 bits d'une représentation brute. Cette mesure borne la longueur moyenne d'un code sans perte qui ne modélise que les fréquences individuelles ; elle ne tient pas compte des dépendances entre octets. Un code de Huffman peut exploiter cette asymétrie, mais un modèle d'ordre zéro ne représente pas directement les séquences récurrentes telles que les mots, les espaces d'indentation, les séparateurs CSV ou les fins de ligne.

Le programme contient des mots-clés, des noms d'identificateurs, des espaces et une ponctuation récurrents, mais aussi des commentaires et des nombres variés. La prose française répète des caractères fréquents et des motifs lexicaux, tout en conservant une distribution presque dépourvue de longues plages d'octets identiques. Le CSV météorologique répète ses en-têtes, identifiants de stations, formats d'horodatage et libellés de conditions ; ses valeurs numériques changent toutefois d'une ligne à l'autre. Ces structures suggèrent une modélisation contextuelle ou par dictionnaire, que Huffman seul ne saisit pas.

TABLE 1 – Caractéristiques mesurées des fichiers, en octets, bits/octet et octets par plage.

Fichier	Taille [octets]	H_0 [bits/octet]	Plage moyenne [octets]	Plage max. [octets]
Code Python	3 844	4,368	1,33	78
Littéraire français	5 092	4,422	1,02	2
Météo CSV	26 515	4,969	1,06	3

Les entropies sont calculées par $H_0 = -\sum_{b=0}^{255} p_b \log_2 p_b$. Elles correspondent à des bornes idéales d'environ 2 098, 2 814 et 16 456 octets respectivement, avant tout coût de dictionnaire, d'en-tête ou d'alignement. Le code source rapporte aussi une « couverture » des motifs de quatre octets de 100 % pour les trois entrées. Cette valeur est plafonnée et additionne des occurrences chevauchantes ; elle ne mesure donc pas la fraction compressible et n'est pas utilisée comme preuve de redondance. Une analyse par phrases LZ non chevauchantes ou par gains réels d'un encodeur serait nécessaire.

2.2 Question 2 : pistes et choix de méthode

Huffman constitue une base pertinente lorsque certaines valeurs d'octets sont plus fréquentes que d'autres ; il reste limité à un modèle de fréquences globales et nécessite de transmettre son arbre ou sa table de codes [2]. Le codage par plages (RLE) serait efficace sur de longues suites identiques, mais les plages observées sont très courtes. Un dictionnaire de type LZ77/LZW pourrait exploiter les mots, lignes et champs récurrents, au prix d'une fenêtre ou d'un dictionnaire et de références plus coûteuses. Une transformation prédictive d'ordre 1, suivie de Golomb [3], est une piste plus simple : si les octets voisins sont corrélés, les résidus se concentrent près de zéro et les quotients de Golomb restent courts.

La méthode testée est donc la prédiction différentielle avec appariement ZigZag, puis le codage de Golomb avec un paramètre m choisi par recherche empirique de 1 à 63. La prédiction utilise la différence entre deux octets successifs ; ZigZag convertit les différences signées en entiers non négatifs. Ce choix est réversible, mais son efficacité dépend fortement d'une distribution de résidus favorable. Les premiers octets textuels et transitions entre mots ou champs peuvent au contraire produire des écarts élevés. Comme les résultats ci-dessous ne satisfont pas le critère de surpasser Huffman sur chaque fichier, cette piste doit être considérée comme une expérimentation non concluante, et non comme une solution répondant à l'objectif.

3 Implémentation et évaluation

3.1 Question 3 : démarche et outils

Le notebook `code.ipynb` lit les données avec `open(..., 'rb')` et calcule les fréquences avec `Counter` et `NumPy` [4]. Il contient une implémentation de Huffman construite à partir des fréquences des symboles, une routine de prédiction ZigZag et un encodeur/décodeur de Golomb. La valeur de m est choisie en minimisant la longueur de la chaîne binaire codée sur les candidats $1 \leq m < 64$. Les distributions et comparaisons sont tracées avec `Matplotlib` [5]. L'implémentation de Huffman s'inspire du notebook de cours référencé en bibliographie.

Les assertions du notebook vérifient que le décodage Huffman et le décodage prédictif-Golomb retrouvent la séquence d'octets d'entrée pour les fichiers testés. C'est un contrôle utile de réversibilité, mais il ne valide pas à lui seul les tailles finales : les codes sont conservés comme chaînes de caractères de bits en mémoire, sans sérialisation d'un flux compact ni transmission effective des paramètres et tables nécessaires au décodage.

La Figure 1 montre les histogrammes des résidus ZigZag du notebook. Les valeurs se concentrent près de zéro, surtout pour le CSV, mais les queues et les pics éloignés indiquent que de nombreux écarts restent coûteux pour un code de Golomb unaire.

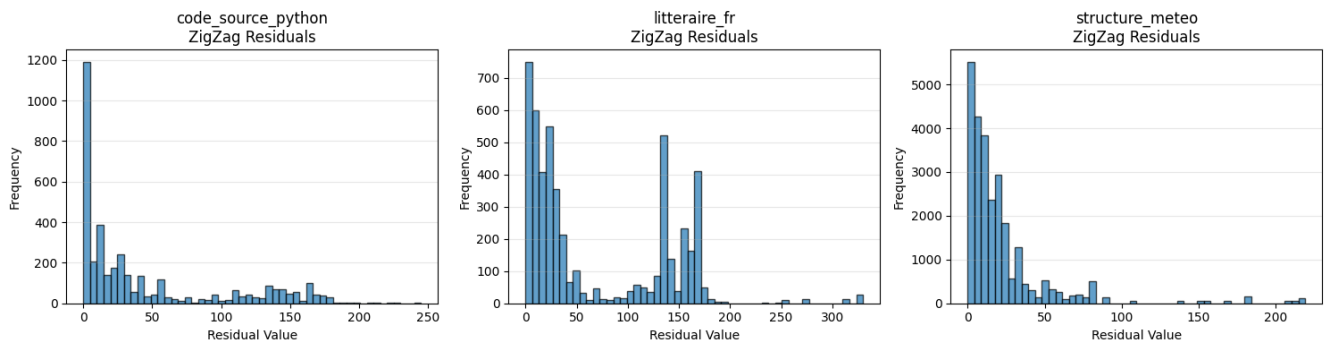


FIGURE 1 – Distribution des résidus ZigZag après prédiction d'ordre 1. L'axe horizontal représente la valeur résiduelle [sans unité] et l'axe vertical le nombre d'octets observés [occurrences].

3.2 Question 4 : résultats et discussion

Le tableau 2 reprend les tailles estimées imprimées par le notebook. Pour Huffman, le calcul divise la longueur des codes par huit puis ajoute une approximation de 4 octets par entrée du dictionnaire. Pour Golomb, il divise le nombre de bits par huit sans coût explicite pour m , le remplissage du dernier octet ou un en-tête. « Golomb + Huffman » applique une seconde transformation à la chaîne de bits Golomb ; le calcul de son coût additionne des dictionnaires estimés. Ces nombres sont donc des comparaisons expérimentales indicatives, pas des tailles de fichiers compressés sérialisés.

TABLE 2 – Tailles estimées par le notebook, en octets, et rapport à la taille originale [%].

Fichier	Original [octets]	Huffman [octets, %]	Golomb prédictif [octets, %]	Golomb + Huffman [octets, %]
Code Python	3 844	2 383 (62,0)	3 279,88 (85,3)	3 559,88 (92,6)
Littéraire français	5 092	3 077,50 (60,4)	4 833,88 (94,9)	5 089,88 (100,0)
Météo CSV	26 515	16 761,38 (63,2)	19 829,62 (74,8)	20 057,62 (75,6)

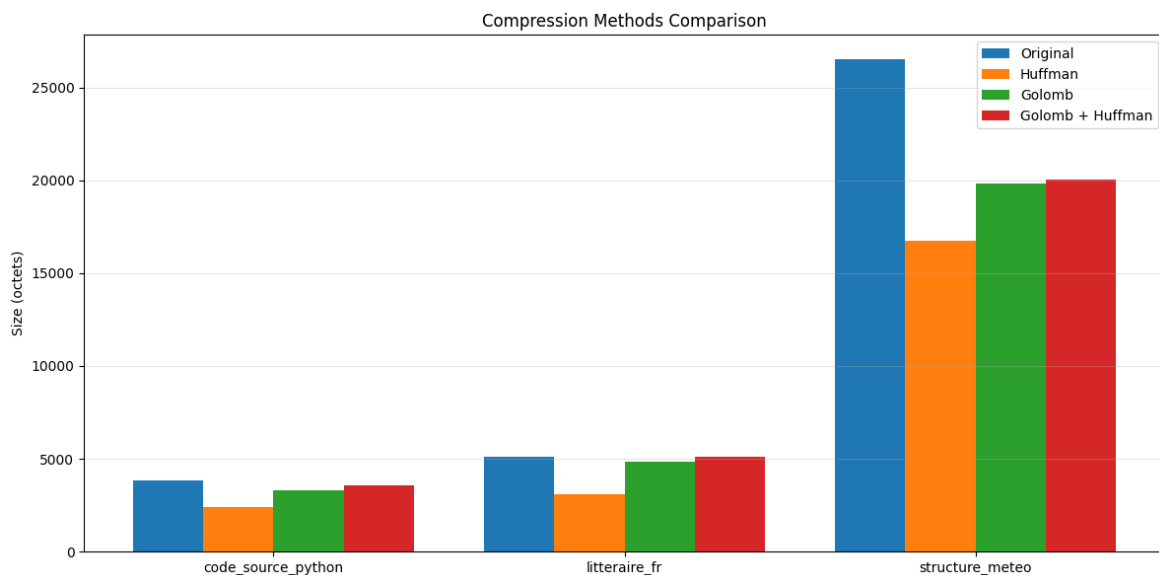


FIGURE 2 – Comparaison des tailles estimées par le notebook pour les trois textes et les méthodes testées [octets]. Les coûts d'en-tête et de sérialisation ne sont pas entièrement inclus.

Selon ces estimations, Huffman est plus compact que la méthode retenue sur les trois entrées. La chaîne prédictive-Golomb conserve respectivement 23,3, 34,5 et 11,6 points de pourcentage de plus que la taille Huffman estimée. Le CSV est le cas le moins défavorable à Golomb : les résidus ZigZag ont une moyenne de 22,02 et 91 valeurs distinctes, contre 69,60 et 163 pour le texte littéraire. Malgré cette réduction de diversité par rapport aux octets originaux, le codage unaire des quotients reste coûteux et annule le bénéfice attendu. Pour la prose, les longues plages répétées sont quasiment absentes ; pour le code, le caractère exceptionnel d'une plage de 78 octets ne suffit pas à compenser les coûts globaux.

L'application d'un Huffman supplémentaire à la suite Golomb n'améliore pas les estimations ; elle ajoute la représentation d'un second dictionnaire sans éliminer le coût de la première transformation. Plus fondamentalement, la condition centrale du TP — battre le Huffman classique sur les trois textes — n'est pas atteinte par la méthode implémentée. Une étape suivante devrait tester une méthode à dictionnaire (par exemple LZ77) sur les mêmes fichiers, comparer les flux réellement sérialisés avec métadonnées incluses, puis vérifier la restitution octet par octet. Le notebook ne mesure pas les temps de compression et décompression ; aucune conclusion de performance temporelle n'est donc possible.

4 Conclusion

L'analyse met en évidence une entropie d'ordre zéro sensiblement inférieure à 8 bits par octet et des motifs répétés propres à chaque type de texte, mais peu de longues plages identiques. Le codage prédictif-Golomb est réversible dans les essais rapportés, cependant ses estimations restent supérieures à la référence Huffman pour le code, la prose et le CSV. L'objectif de compression du TP n'est ainsi pas démontré. En outre, l'absence de flux binaires sérialisés signifie que les ratios du tableau ne sont pas des ratios de fichiers finaux. Une conclusion définitive exige une implémentation avec format de sortie explicite, coût réel des tables et paramètres, mesures de temps et tests de reconstruction bit à bit ; une méthode par dictionnaire constitue une piste mieux adaptée aux répétitions lexicales et structurelles observées.

Références

- [1] G.-A. Bilodeau, *INF8770 – Technologies multimédias, dépôt de cours : notebook de codage de Huffman*, GitHub, <https://github.com/gabilodeau/INF8770>. Code adapté pour le notebook du TP.
- [2] D. A. Huffman, « A Method for the Construction of Minimum-Redundancy Codes », *Proceedings of the IRE*, vol. 40, no. 9, p. 1098–1101, 1952, doi :10.1109/JRPROC.1952.273898.
- [3] S. W. Golomb, « Run-Length Encodings », *IEEE Transactions on Information Theory*, vol. 12, no. 3, p. 399–401, 1966, doi :10.1109/TIT.1966.1053907.
- [4] NumPy Developers, *NumPy documentation*, <https://numpy.org/doc/>.
- [5] Matplotlib Developers, *Matplotlib documentation*, <https://matplotlib.org/stable/>.